Assignment 2 Report
CS683 - ACA

Mangesh, 25M0831
Sourabh, 26M0783
Kumar Rajnish, 25M2123

Q1)

We studied the provided IP-stride prefetcher implementation in prefetcher/ip_stride.l1d_pref. It tracks memory access patterns for individual instruction pointers (IPs) and calculates the stride between consecutive accesses. When a consistent stride is detected, it predicts future memory addresses and issues prefetch requests to reduce cache misses and improve performance.

A) Table :

Trace1

	Baseline	**Ip_Stride**	**SPP**
IPC	0.262756	0.264472	**0.264976**
Speedup	1.0	1.006531	**1.008449**
L1D Load MPKI	**9.54228**	8.91764	8.72932
Prefetch accuracy(%)	–	56.8693	**62.9252**
Coverage(%)	0.0	6.546025	**8.519557**

Trace2

	Baseline	**Ip_Stride**	**SPP**
IPC	0.275099	0.275206	**0.276440**
Speedup	1.0	1.000389	**1.004875**
L1D Load MPKI	**10.91020**	10.83210	10.61390

Prefetch accuracy(%)	—	38.4916	**62.9341**
Coverage(%)	0.0	0.715661	**2.715624**

Trace3

	Baseline	**Ip_Stride**	**SPP**
IPC	0.282671	0.284427	**0.285107**
Speedup	1.0	1.006212	**1.008618**
L1D Load MPKI	**8.00216**	7.59252	7.30272
Prefetch accuracy(%)	–	31.5916	**44.2718**
Coverage(&)	0.0	5.119118	**8.740640**

B) Total Storage Overhead:

Structure	**Field Types & Composition**	**Entry Count**	**Size (Bytes)**
IP Entry	7 × 64-bit fields, 3 × 8-bit fields, 1 × 32-bit field, 1 × bool, history[3]	64	4608
Signature Entry	2 × 64-bit fields, 1 × 8-bit field, 1 × bool	1024	24576
Tracker Entry	1 × 64-bit field, 1 × bool	4096	65536
Total Table Storage	**–**	**–**	**90,112 (90KiB)**

C) Threshold Accuracy value selected:
constexpr double ACCURACY_HIGH = 0.60;
constexpr double ACCURACY_LOW = 0.25;

Reason:

- **>= 60%:Increase degree** - High usefulness suggests the prefetcher can try issuing more prefetches.
- **25% to 60%:Maintain degree** - Avoid changing aggressiveness when accuracy is moderate.
- **<25%:Decrease degree** - Low usefulness suggests reducing potentially wasteful prefetch requests.

These heuristic selected threshold were selected with intention to balance **Performance** - issue enough prefetches to bring data before demand and **Efficiency** - avoids unnecessary requests that consume memory bandwidth and cache capacity.

We also experimented with different accuracy-threshold values, but observed no significant changes in the performance metrics. Therefore, we retained 60% and 25% as heuristic thresholds.

D)

- **>= 60% : Increase degree by 1, up to 4**
- **25% to 60% : Keep the current degree**
- **<25% : Decrease degree by 1, down to 1**

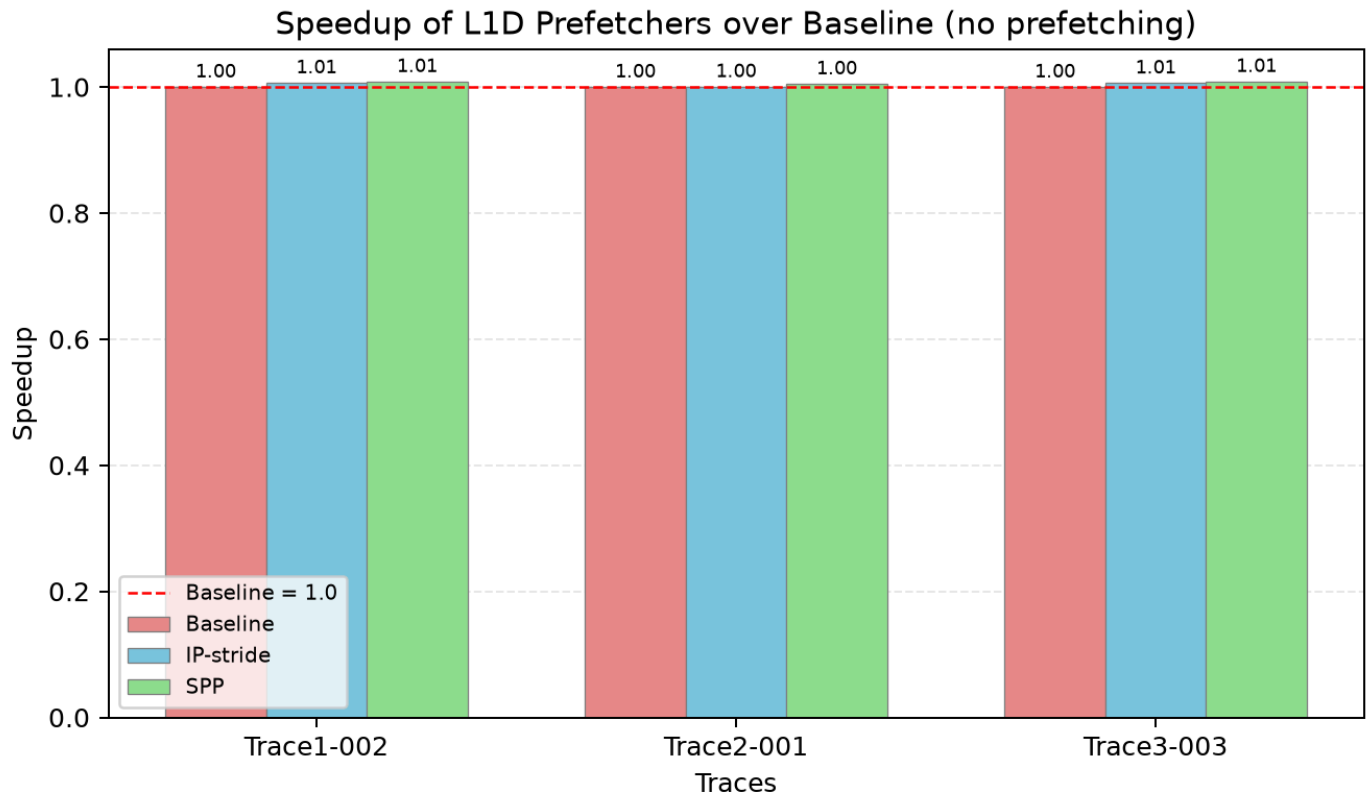
The degree starts at 2, adjusts gradually by one step, and stays within 1–4.

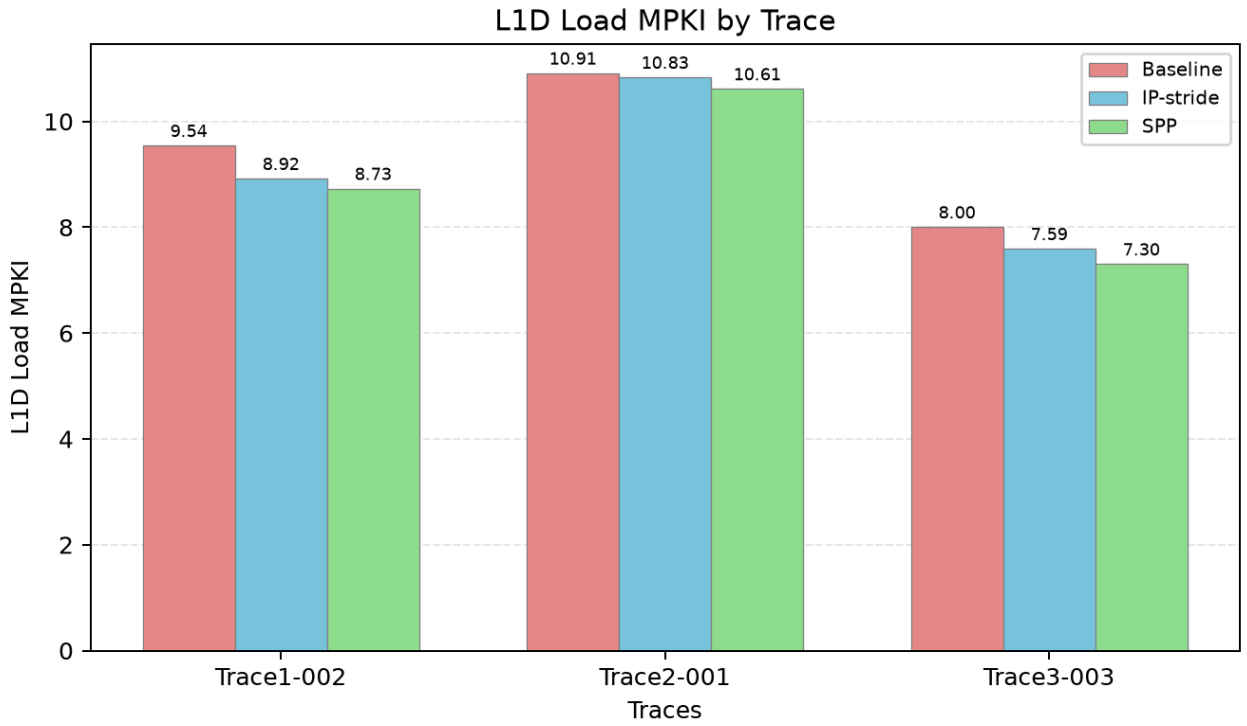
Justification:
We selected a prefetch degree range of 1–4, with an initial degree of 2, to balance conservative and aggressive prefetching. We experimented SPP for degree 1, 2, 3 and 4. In our experiments, degree 4 achieved the highest IPC and lowest L1D load MPKI across all three traces. On Trace1-002, speedup improved from **1.0075× (degree 1) to 1.0084× (degree 4)**, while MPKI decreased

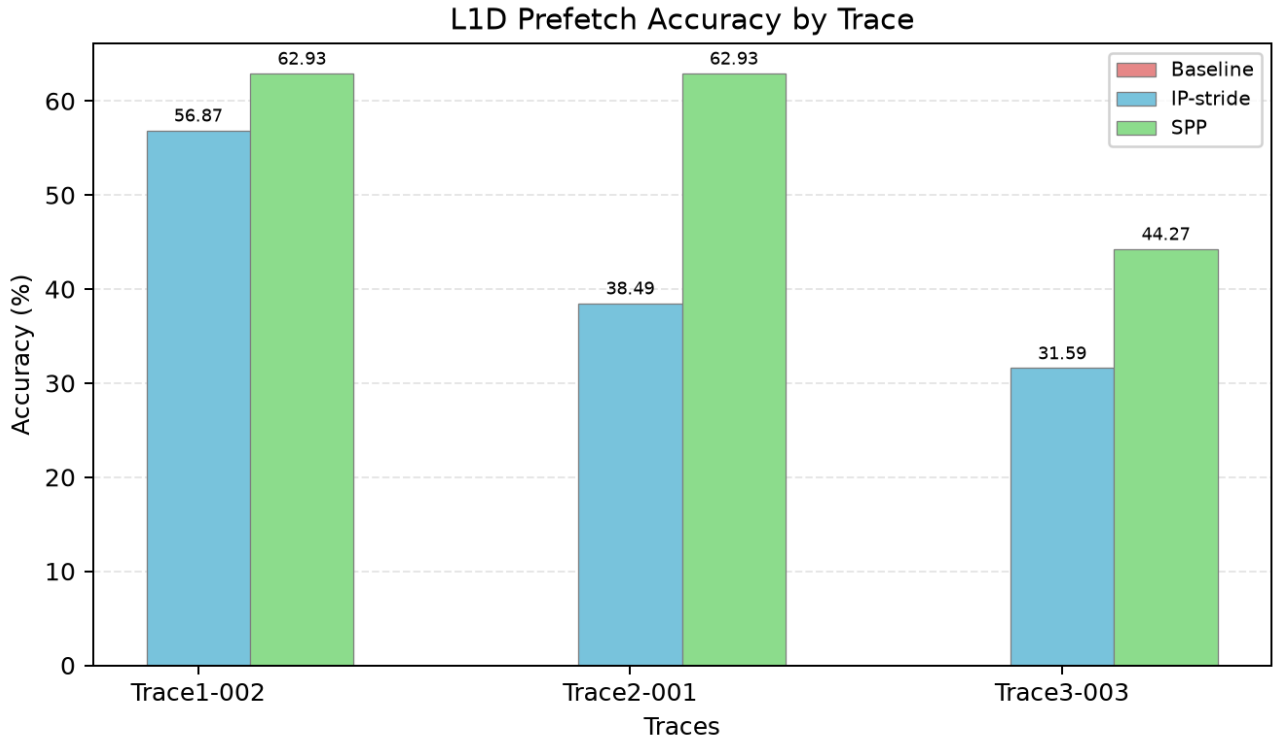
from **8.8625 to 8.7293**. On Trace2-001, speedup improved from **1.0030× to 1.0049×**, and on Trace3-003, from **1.0075× to 1.0086×**. However, prefetch accuracy decreased with increasing degree on the first two traces. These results motivate a bounded degree range that allows the prefetcher to adjust its aggressiveness according to workload behavior.

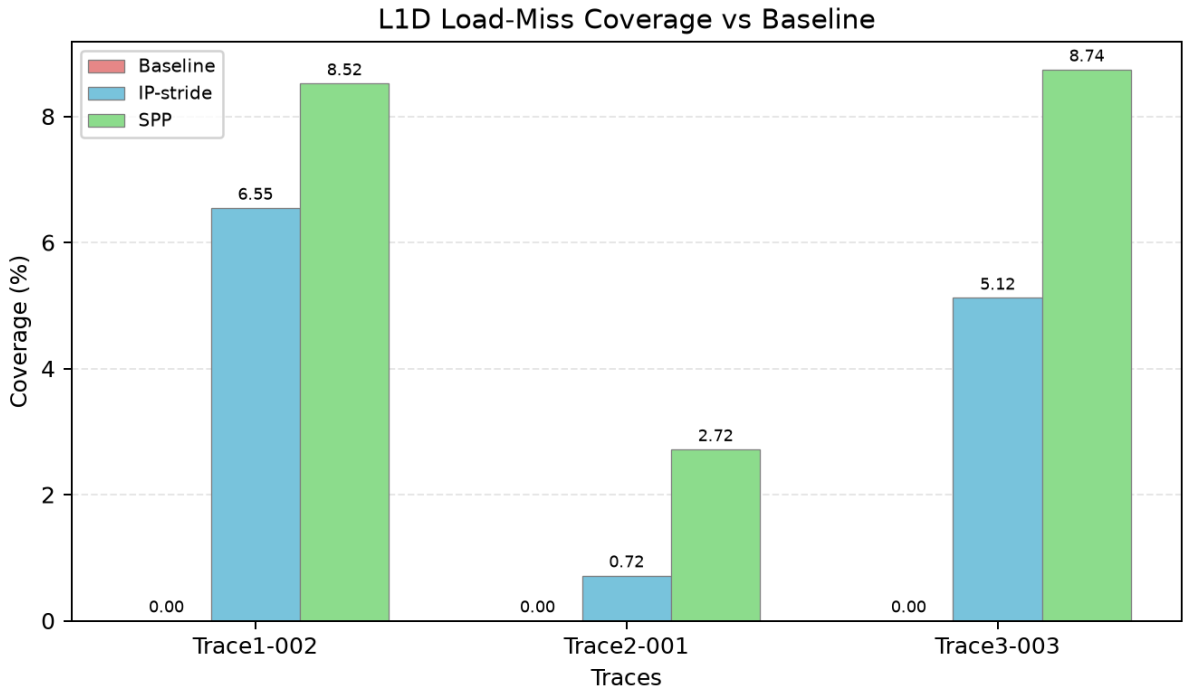
Trace	Degree	Speedup	L1D load MPKI	Accuracy
Trace1-002	1	1.0075×	8.8625	67.10%
Trace1-002	2	1.0075×	8.8625	67.10%
Trace1-002	3	1.0081×	8.7842	65.00%
Trace1-002	4	1.0084×	8.7293	62.93%
Trace2-001	1	1.0030×	10.6870	65.67%
Trace2-001	2	1.0030×	10.6870	65.67%
Trace2-001	3	1.0036×	10.6546	64.11%
Trace2-001	4	1.0049×	10.6139	62.93%
Trace3-003	1	1.0075×	7.5347	38.85%
Trace3-003	2	1.0075×	7.5347	38.85%
Trace3-003	3	1.0084×	7.4529	39.24%
Trace3-003	4	1.0086×	7.3027	44.27%

Plots:


Speedup of L1D Prefetchers over Baseline (no prefetching)
Speedup
Traces
Trace1-002
Trace2-001
Trace3-003
Baseline = 1.0
Baseline
IP-stride
SPP
1.00
1.01
1.01
1.00
1.00
1.00
1.00
1.01
1.01


L1D Load MPKI by Trace
L1D Load MPKI
Traces
Trace1-002
Trace2-001
Trace3-003
Baseline
IP-stride
SPP
9.54
8.92
8.73
10.91
10.83
10.61
8.00
7.59
7.30


L1D Prefetch Accuracy by Trace
Baseline
IP-stride
SPP
Accuracy (%)
0
10
20
30
40
50
60
56.87
62.93
38.49
62.93
31.59
44.27
Trace1-002
Trace2-001
Trace3-003
Traces

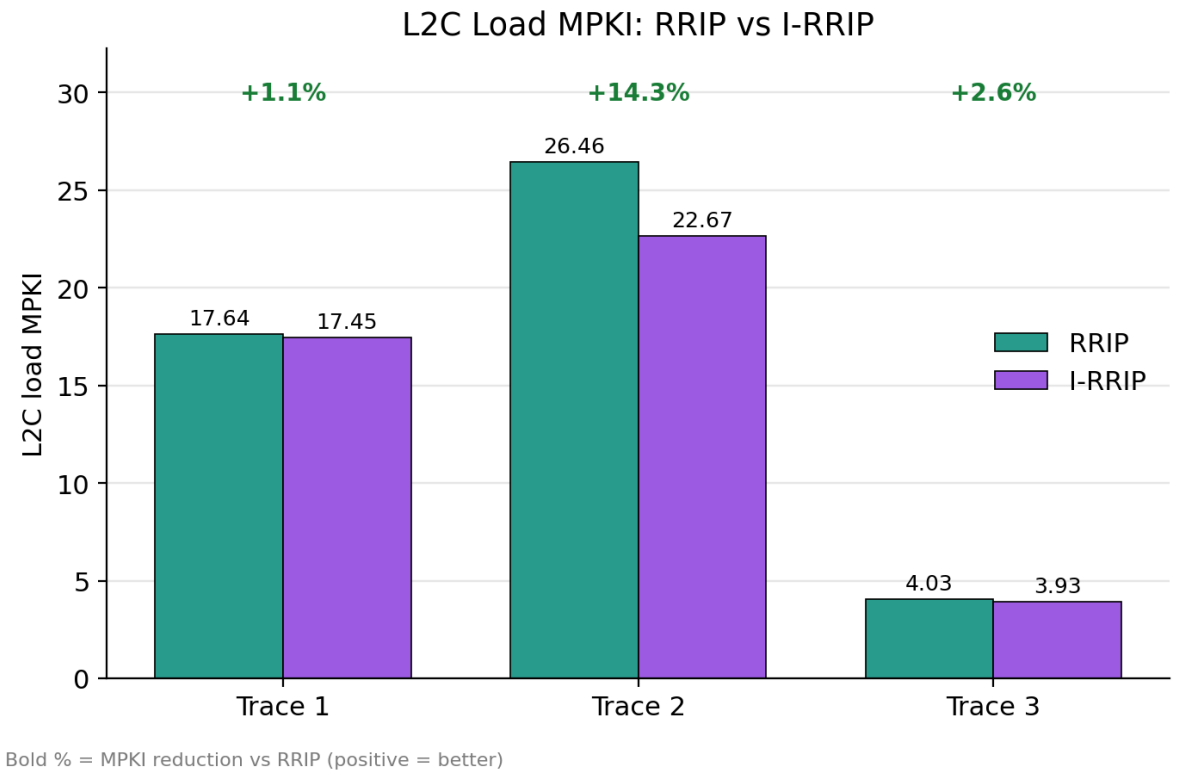

L1D Load-Miss Coverage vs Baseline
Baseline
IP-stride
SPP
Coverage (%)
0
2
4
6
8
0.00
6.55
8.52
0.00
0.72
2.72
0.00
5.12
8.74
Trace1-002
Trace2-001
Trace3-003
Traces

Q2)

A.) Instruction-Conscious L2 Replacement Policy (I-RRIP)

This is suitable for a large instruction footprint workload. We kept the 2-bit RRPV structure and made it instruction-aware, using one extra bit per line that records whether the line holds code.

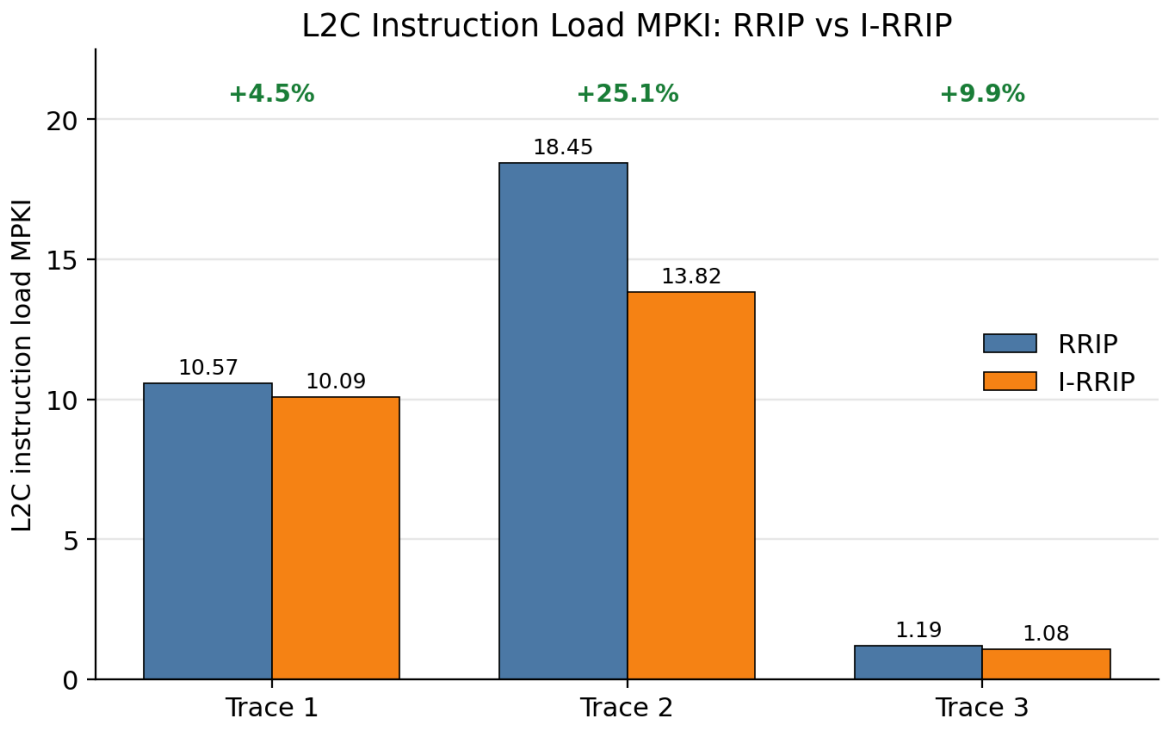
Insertion (unchanged) = Both instruction and data fills still go in at RRPV 2, exactly like SRRIP. We deliberately did not give instructions a better insertion value. At fill time we can't tell if a code line will be reused or is a one-off, and inserting every code line near RRPV 0 would let single-use code push out useful data. We let a line earn its protection on its first hit instead.



Bold % = MPKI reduction vs RRIP (positive = better)

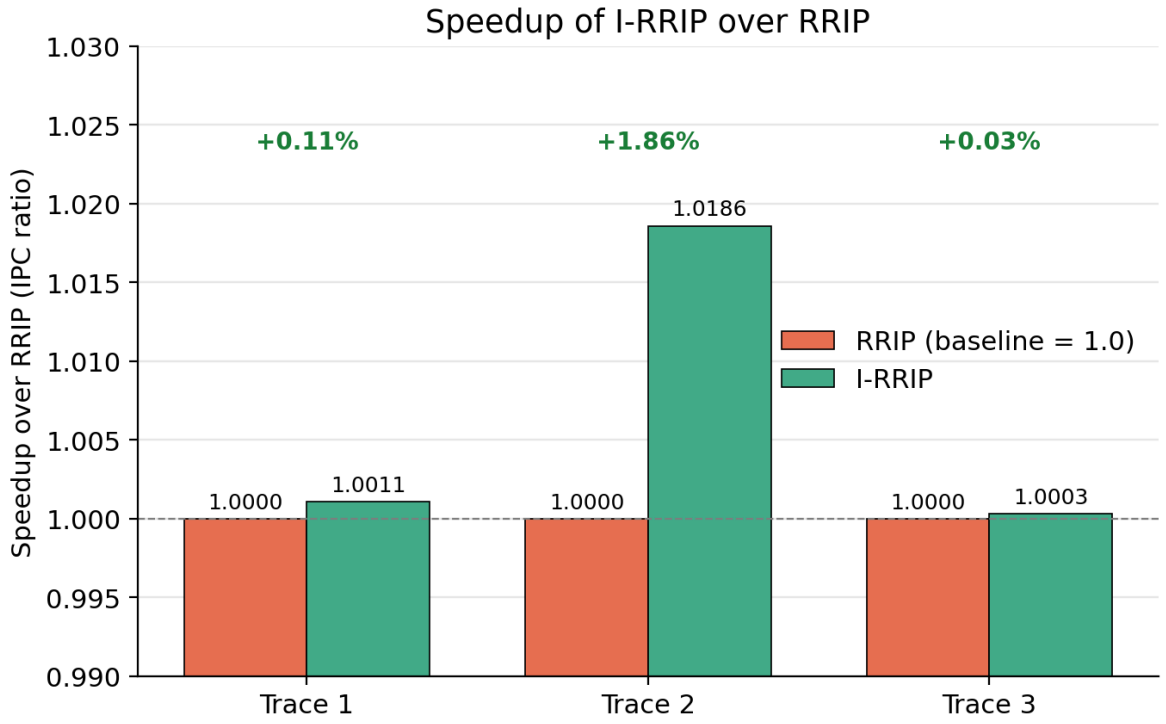
Eviction (changed) = Among lines at RRPV 3, we evict a data line first. An instruction line is evicted only when no data line is at max, so stale code can still leave. Without that fallback, dead instruction lines could build up and starve the set.

Trace	Metric	RRIP	I-RRIP	Change
T1	IPC	0.231983	0.232232	+0.107% speedup
T1	L2C Load MPKI	17.6424	17.4518	+1.08%
T1	L2C Instrn MPKI	10.5698	10.0897	+4.54%
T1	L2C Data MPKI	7.07256	7.36208	−4.09%
T2	IPC	0.251539	0.256213	+1.858% speedup
T2	L2C Load MPKI	26.4582	22.6667	+14.33%
T2	L2C Instrn MPKI	18.4474	13.8182	+25.09%
T2	L2C Data MPKI	8.0108	8.84852	−10.46%
T3	IPC	0.381892	0.382014	+0.032% speedup
T3	L2C Load MPKI	4.03464	3.93016	+2.59%
T3	L2C Instrn MPKI	1.19492	1.0764	+9.92%
T3	L2C Data MPKI	2.83972	2.85376	−0.49%



Bold % = MPKI reduction vs RRIP (positive = better)

Promotion (changed) = An instruction hit still goes straight to RRPV 0, as in RRIP. A data hit now only lowers RRPV by 1 instead of resetting to 0. A line that has already shown reuse is likely to be reused again, and the gentler data promotion keeps data from quickly outranking code.



Speedup = IPC(I-RRIP) / IPC(RRIP). Y-axis starts at 0.99 to make small differences visible.

Trade-offs = Instruction MPKI fell on every trace, while data MPKI rose on every trace. The effect is strongest on T2, which has the heaviest instruction pressure, so protecting code costs data there. The net IPC is still positive, because an instruction miss costs more than a data miss.

B.) Dynamic I-RRIP

I-RRIP is the static policy from Task 2A, and "D-IRRIP" is the set-dueling policy with threshold x. Speedup is IPC / IPC(RRIP). MPKI percentages are reductions versus RRIP, so a positive value is better and a negative value is worse. The dynamic runs used 32 leader sets (one leader per 32 sets), an epoch of 2048 leader events and the score (instr. accesses + instr. misses) / (total accesses + total misses).

Score = $(instr.\ accesses + instr.\ misses)/(total\ access + total\ misses)$

This score is used to compare with the threshold and choose a replacement policy.

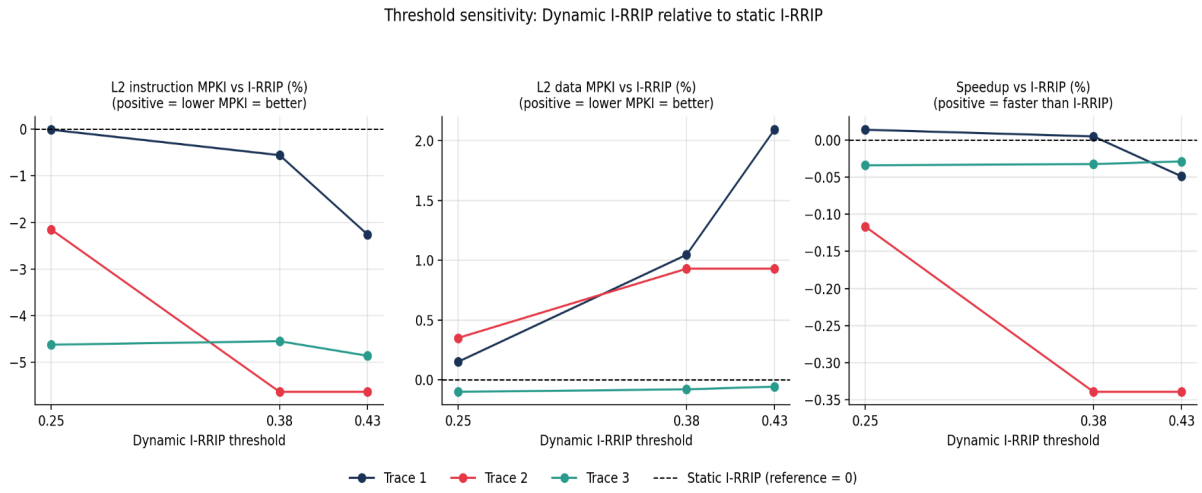
D-IRRIP with threshold value 0.25

Trace	Metric	IRRIP	D-IRRIP	Change
T1	IPC	0.232232	0.232265	+0.014% speedup
T1	L2C Load MPKI	17.4518	17.4418	+0.06%
T1	L2C Instrn MPKI	10.0897	10.0909	−0.01%
T1	L2C Data MPKI	7.36208	7.35092	+0.15%
T2	IPC	0.256213	0.255915	−0.116%
T2	L2C Load MPKI	22.6667	22.9327	−1.17%
T2	L2C Instrn MPKI	13.8182	14.1152	−2.15%
T2	L2C Data MPKI	8.84852	8.81744	+0.35%
T3	IPC	0.382014	0.381885	−0.034%
T3	L2C Load MPKI	3.93016	3.98272	−1.34%
T3	L2C Instrn MPKI	1.0764	1.12612	−4.62%
T3	L2C Data MPKI	2.85376	2.8566	−0.10%

Rational for 32 Leader sets = One leader every 32 sets samples the whole index range, so the estimate is not biased towards one region of memory. More leaders strand more sets on RRIP. Fewer give a noisier score and fewer decisions per run. 32 (3.1%) is a middle point, and it costs only a small slice of I-RRIP's gain. Leaders always run RRIP so that the statistics describe the workload

Sensitivity Analysis of multiple Thresholds = Lower threshold is better, and it never beats static I-RRIP by a meaningful margin. Moving the threshold from 0.25 to 0.43 steadily gives up I-RRIP's instruction gain.

Trace 2 is where the policy matters Instruction MPKI goes 13.82 (I-RRIP), 14.12 (0.25), 14.60 (0.38 and 0.43), 18.45 (RRIP). The dynamic policy lands between the two static policies, so it spends part of the run in RRIP mode even though I-RRIP is better throughout. The runs at 0.38 and 0.43 are identical. That means no epoch had a score between 0.38 and 0.43, so both thresholds made the same decision in every epoch.



Trace 3 barely responds to the threshold Instruction MPKI is 1.126, 1.125 and 1.129, a spread of 0.4%, and it keeps only about 56-59% of I-RRIP's gain at every threshold.

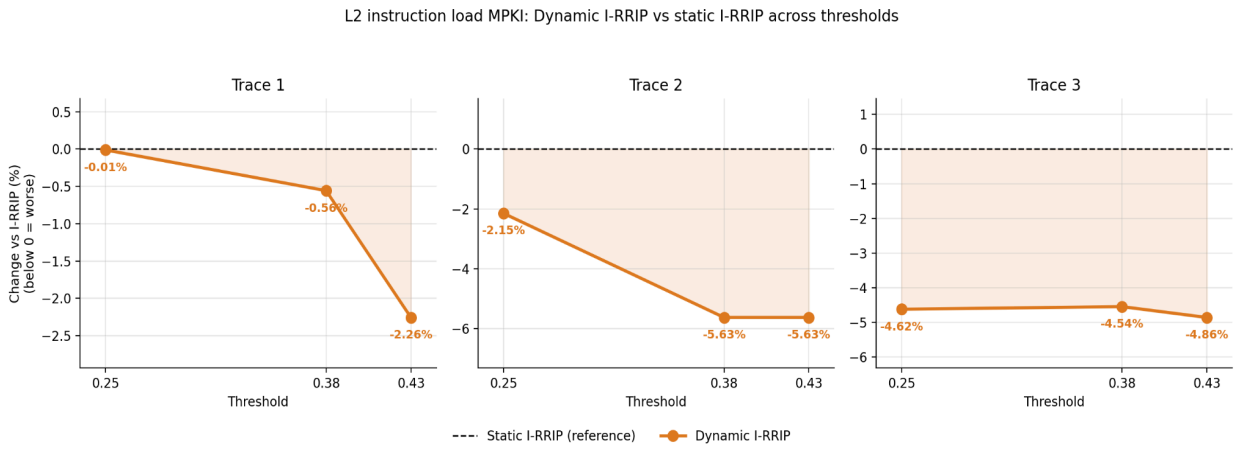
Data MPKI moves the other way Anything that spends more time in RRIP mode recovers some data MPKI. The instruction loss is larger than the data gain on every trace, and because instruction misses stall the front end, IPC follows instruction MPKI.

D-IRRIP with threshold value 0.38

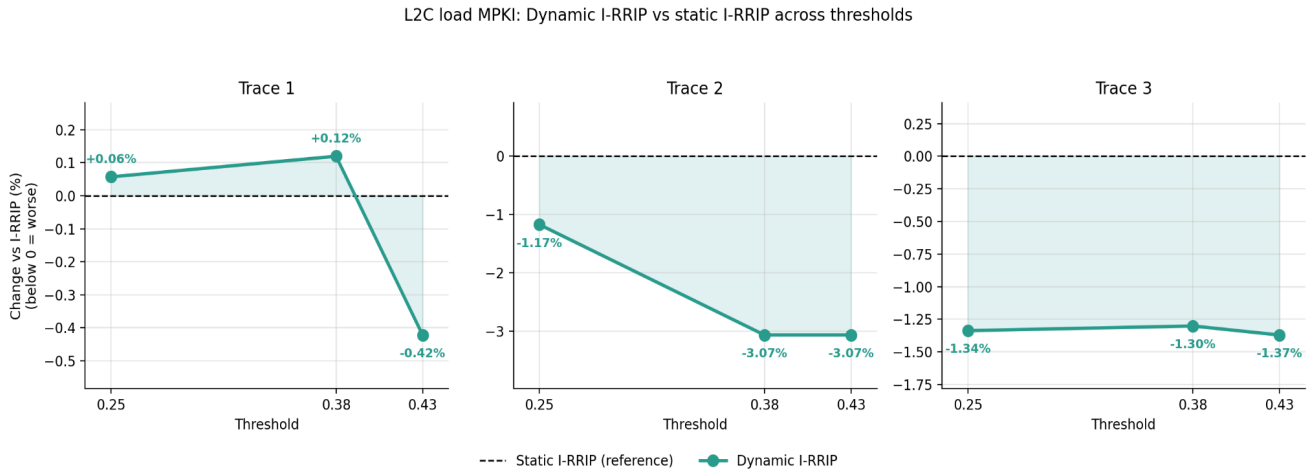
Trace	Metric	IRRIP	D-IRRIP	Change
T1	IPC	0.232232	0.232244	+0.005% speedup
T1	L2C Load MPKI	17.4518	17.4309	+0.12%
T1	L2C Instrn MPKI	10.0897	10.1459	−0.56%
T1	L2C Data MPKI	7.36208	7.285	+1.05%
T2	IPC	0.256213	0.255343	−0.340%
T2	L2C Load MPKI	22.6667	23.3624	−3.07%
T2	L2C Instrn MPKI	13.8182	14.5962	−5.63%
T2	L2C Data MPKI	8.84852	8.7662	+0.93%
T3	IPC	0.382014	0.381891	−0.032%
T3	L2C Load MPKI	3.93016	3.98132	−1.30%
T3	L2C Instrn MPKI	1.0764	1.12532	−4.55%
T3	L2C Data MPKI	2.85376	2.856	−0.08%

Why the dynamic policy falls short of static I-RRIP –

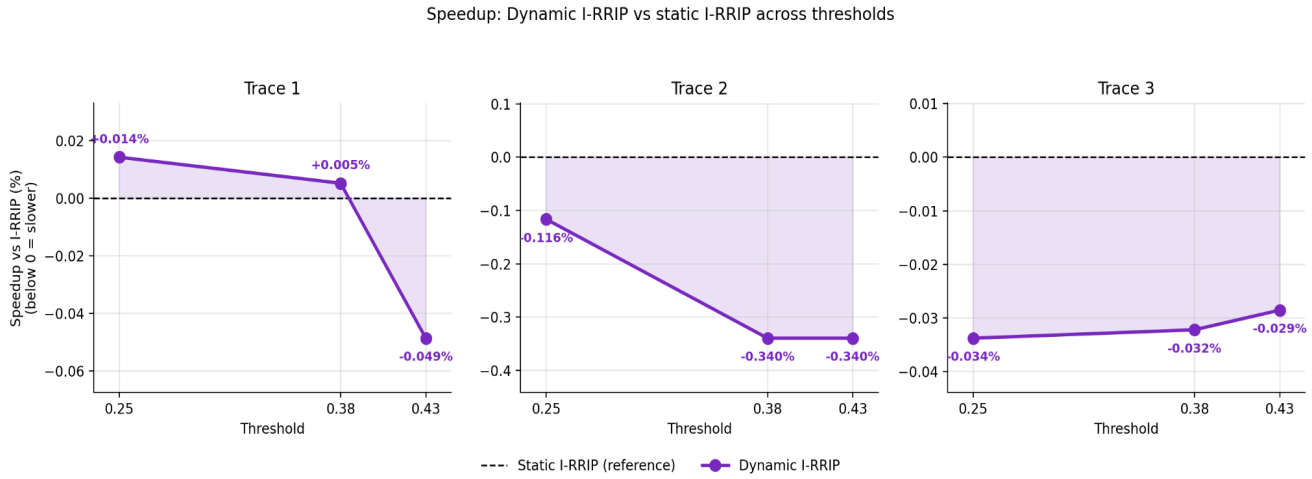
Leader sets are permanently on RRIP 32 of 1024 sets (about 3.1%) never get I-RRIP. If I-RRIP's benefit is spread evenly across sets, this alone costs up to about 3% of its gain. On Trace 2 that is roughly 0.14 MPKI of the 0.30 MPKI gap between 0.25 and static I-RRIP, so about half the gap is the cost of measuring, and the rest is epochs spent in RRIP mode.



The score that we used is the share of instruction traffic in the RRIP leader sets. It says how much code there is, not whether protecting it pays off right now. Changing the parameters in score might improve the MPKI.



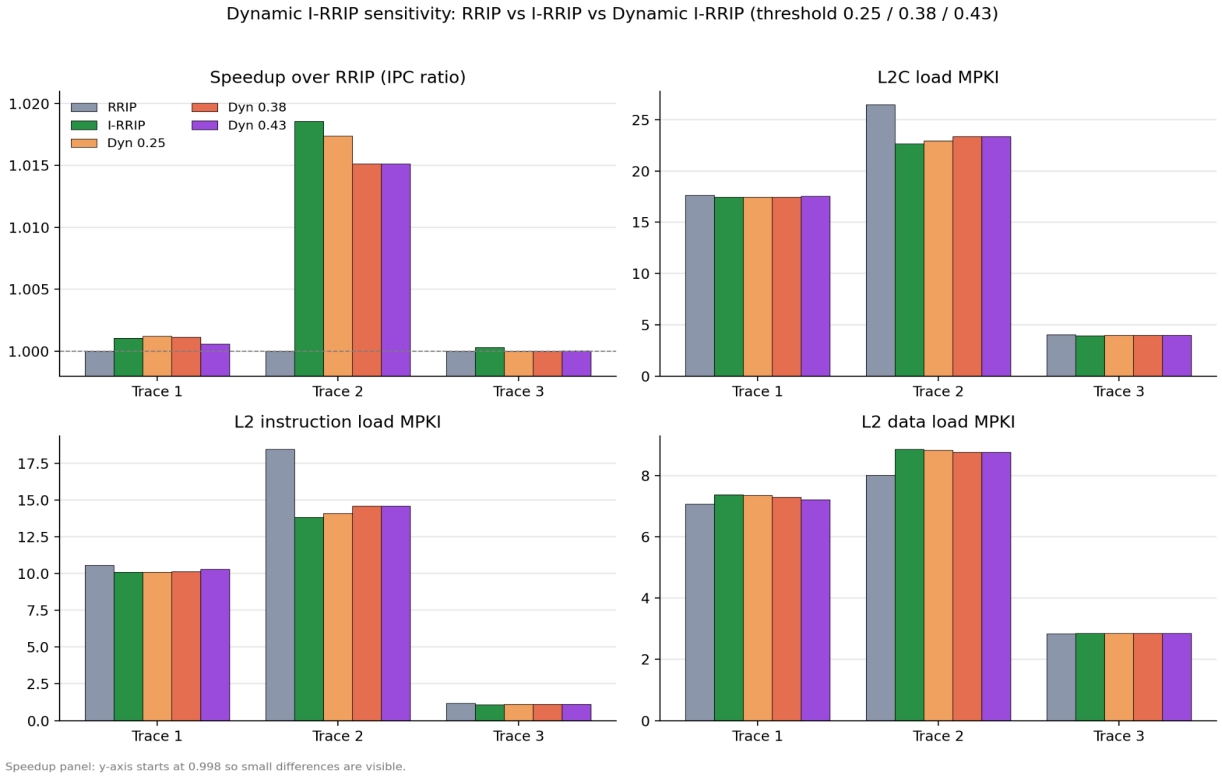
The policy chooses between RRIP and I-RRIP, and I-RRIP was at least as good on every trace in IPC and better in instruction MPKI. This is why every dynamic run trails static I-RRIP.



Conclusion of analysis = The score combines access (how much code the L2 handles) and misses (how costly it is). If instructions are a small share of the traffic, protecting them is not worth the data penalty (data MPKI rises 0.5-10.5% under I-RRIP), so followers fall back to RRIP. The results above show the limits of this design: on these traces its best setting to the lowest threshold, where it behaves almost like static I-RRIP.

D-IRRIP with threshold value 0.43

Trace	Metric	IRRIP	D-IRRIP	Change
T1	IPC	0.232232	0.232119	−0.049%
T1	L2C Load MPKI	17.4518	17.5254	−0.42%
T1	L2C Instrn MPKI	10.0897	10.3174	−2.26%
T1	L2C Data MPKI	7.36208	7.208	+2.09%
T2	IPC	0.256213	0.255343	−0.340%
T2	L2C Load MPKI	22.6667	23.3624	−3.07%
T2	L2C Instrn MPKI	13.8182	14.5962	−5.63%
T2	L2C Data MPKI	8.84852	8.7662	+0.93%
T3	IPC	0.382014	0.381905	−0.029%
T3	L2C Load MPKI	3.93016	3.98404	−1.37%
T3	L2C Instrn MPKI	1.0764	1.12868	−4.86%
T3	L2C Data MPKI	2.85376	2.85536	−0.06%

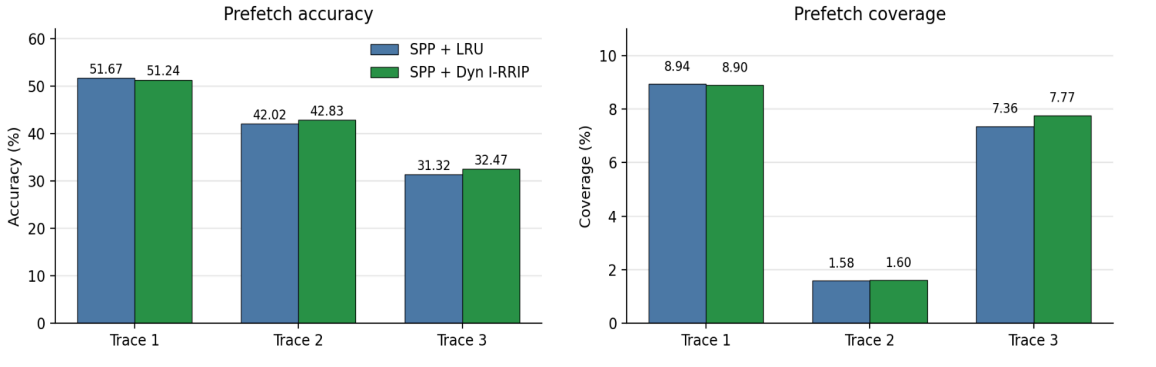


Q3) Incorporating SPP with Dynamic I-RRIP

We combined the **Stride Pattern Prefetcher (SPP)** with **Dynamic Instruction-RRIP (I-RRIP)** to improve cache performance. SPP predicts future memory accesses based on stride patterns and prefetches data before it is needed, while Dynamic I-RRIP manages cache replacement to retain useful instruction lines and reduce cache misses. We also modified the replacement policy to insert prefetched lines with the maximum RRPV, making them easier to evict if they are not used. This helps reduce cache pollution from unnecessary prefetches.

Trace	Config	IPC	Speedup	PF Accuracy	PF Coverage	L1D Load MPKI	L2C Load MPKI	L2 Instr MPKI	L2 Data MPKI
1	No prefetch + LRU	0.2321	1.0000	-	-	10.6781	17.7528	10.4106	7.3422
1	Dyn I-RRIP	0.2323	1.0008	-	-	10.6784	17.4418	10.0909	7.3509
1	SPP + LRU	0.2337	1.0068	51.67%	8.94%	9.7240	16.9648	10.4396	6.5253
1	SPP + Dyn I-RRIP	0.2340	1.0082	51.24%	8.90%	9.7272	16.6042	10.0684	6.5358
2	No prefetch + LRU	0.2508	1.0000	-	-	13.5356	28.1187	19.5200	8.5986
2	Dyn I-RRIP	0.2559	1.0202	-	-	13.5408	22.9327	14.1152	8.8174
2	SPP + LRU	0.2512	1.0014	42.02%	1.58%	13.3224	28.2084	19.7393	8.4691
2	SPP + Dyn I-RRIP	0.2566	1.0231	42.83%	1.60%	13.3196	22.7044	14.0842	8.6202
3	No prefetch + LRU	0.3822	1.0000	-	-	3.4033	3.9533	1.1230	2.8302
3	Dyn I-RRIP	0.3819	0.9992	-	-	3.4032	3.9827	1.1261	2.8566
3	SPP + LRU	0.3847	1.0065	31.32%	7.36%	3.1529	3.5429	1.1372	2.4058
3	SPP + Dyn I-RRIP	0.3846	1.0064	32.47%	7.77%	3.1387	3.5278	1.1205	2.4073

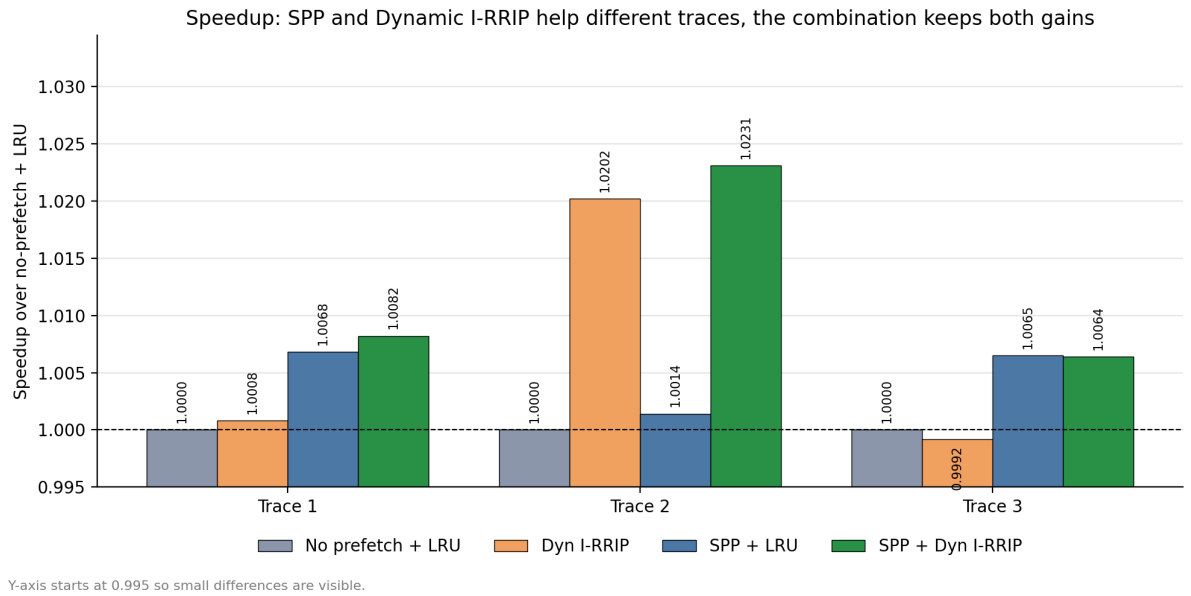
SPP gain follows coverage, and Dynamic I-RRIP nudges accuracy and coverage up on Traces 2 and 3



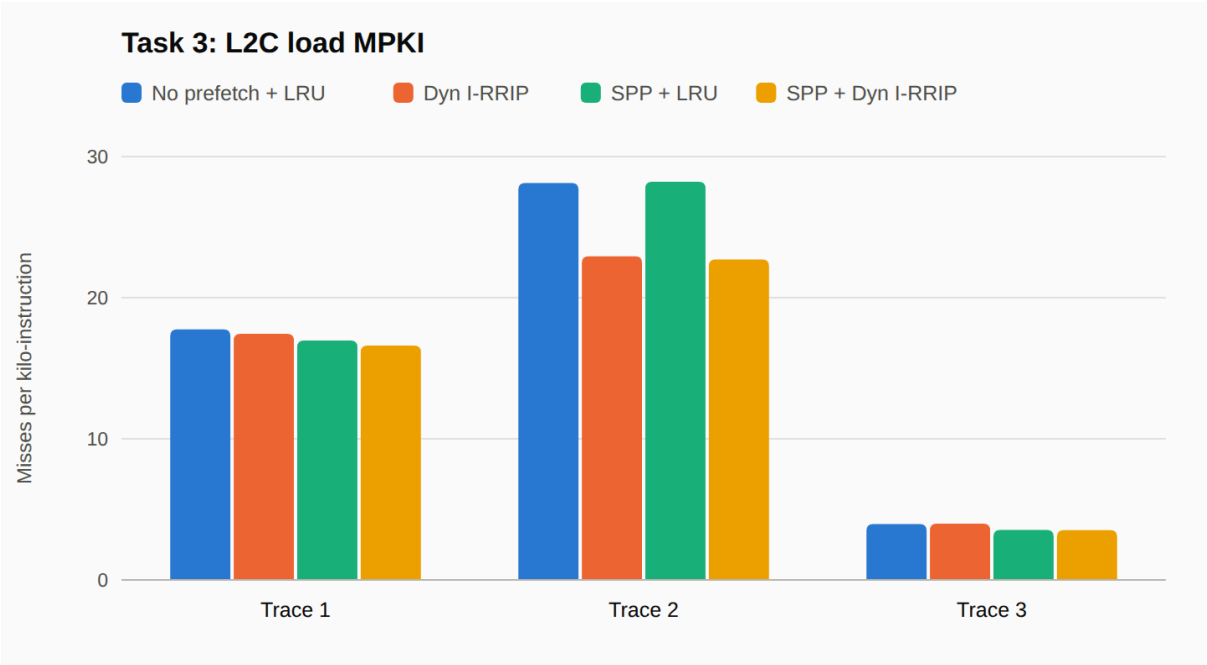
SPP-only speedup: Trace 1 +0.68% Trace 2 +0.14% Trace 3 +0.65% (tracks coverage)

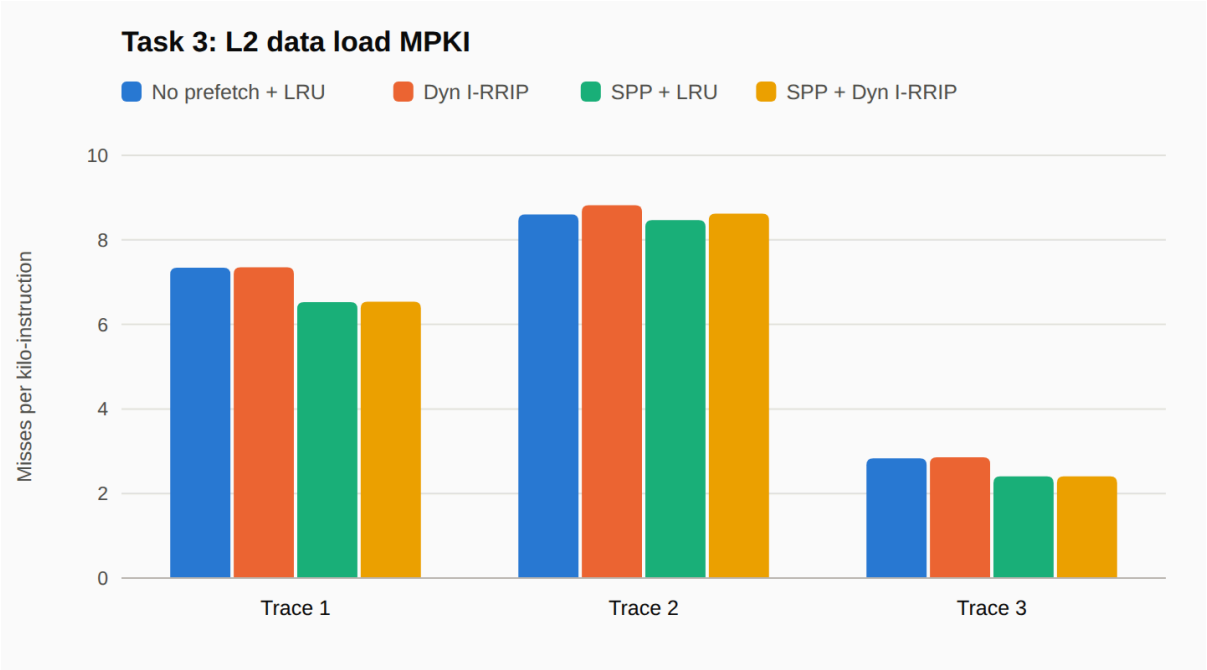
The results show improvements over the baseline of **0.82% on Trace 1 and 2.31% on Trace 2**, while Trace 3 improves by 0.64%. Compared with SPP +

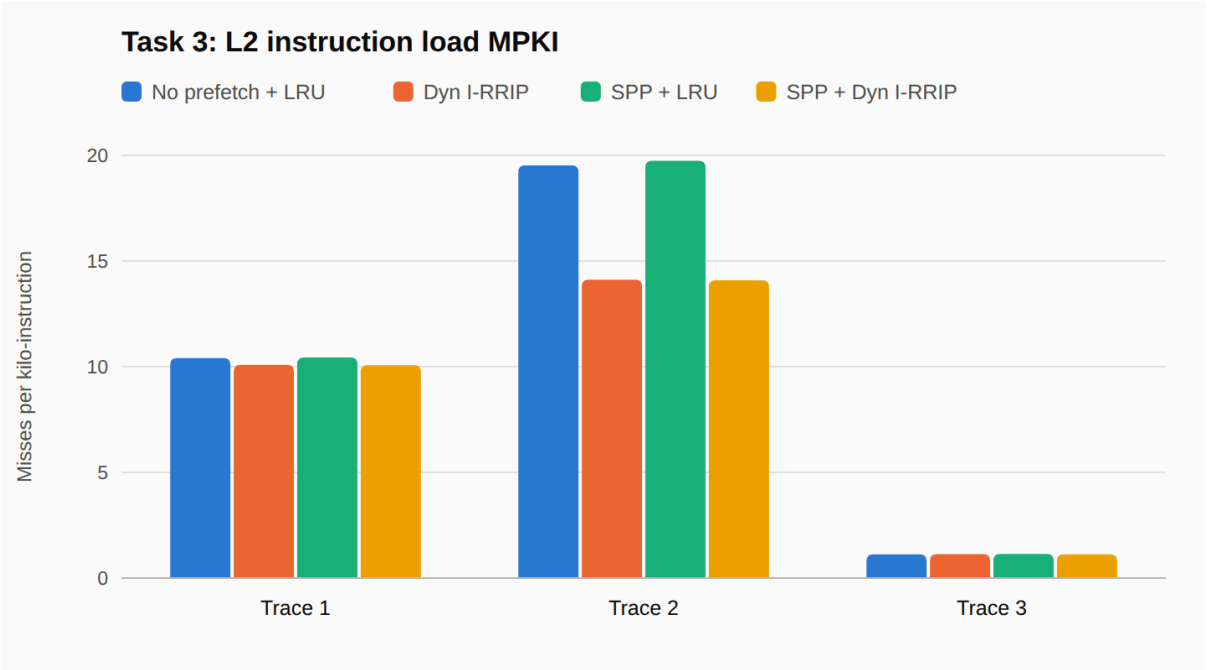
LRU, the combined approach performs better on Traces 1 and 2 but is marginally worse on Trace 3. The improvement is not consistent across all traces because prefetch effectiveness and instruction/data access patterns vary between workloads. Overall, combining prefetching with instruction-aware replacement improves performance on most of the tested traces, although the gains depend on the workload.



Rationale: Prefetched lines are not guaranteed to be used. Inserting them at the maximum RRPV prevents them from receiving the same initial protection as demand-accessed lines, reducing the risk of evicting useful cache blocks.







The combined policy also reduces L2C load MPKI compared with standalone SPP + LRU on every trace. Its effect on prefetch accuracy and coverage varies by workload, indicating that the effectiveness of prefetching depends on the memory-access patterns in each trace.